CHAPTER 5 — SOFTWARE DESIGN

Applied Engineering Scenarios

Section 1: Architectural Strategy Selection & Modularity Evaluation

Q5.S01. Textbook Exercise 5.12: Design Strategy for Reusing Existing Modules

> **Original Textbook Question (Exercise 5.12):**
>
> **If some existing modules are to be re-used in building a new system, which design strategy is used and why?**

> **The design strategy used is Bottom-Up Design.**
>
> **Engineering Rationale & Justification:**
>
> 1. **Starting with Pre-Existing Low-Level Components:** Bottom-up design fundamentally begins at the lowest level of abstraction by identifying, collecting, and organizing the basic building blocks, pre-written library routines, and existing modules that are already implemented and tested.
> 2. **Iterative Composition Upward:** In this strategy, existing modules are collected together in the form of a component repository or "library". Higher-level sub-routines, managers, and functional subsystems are then systematically designed and constructed on top of these pre-existing lower-level modules, assembling upward until the entire system hierarchy is completed.
> 3. **Elimination of Redundant Development:** Reusing existing modules via a bottom-up approach eliminates the need to specify and implement fundamental operational routines from scratch, leveraging concrete, verified software assets.
>
> *(Architectural Nuance in Large Systems: When building large commercial applications where brand-new business requirements must be reconciled with existing module libraries, engineers apply a Hybrid Design strategy—using top-down decomposition to partition high-level requirements while bottom-up integrating the existing library modules, as noted in §5.3.3 / Slide 35. However, the foundational design strategy defined by starting from existing lower-level modules is strictly Bottom-Up Design).*
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–214; Companion Slide 33. Exercise 5.12*

Q5.S02. Enterprise Architectural Scenario: Applied Strategy Selection for Complex Supply-Chain Systems

> **Engineering Scenario:**
>
> An engineering team is contracted to design and construct an enterprise automated inventory, supply-chain, and billing system for an international logistics provider. The organization already possesses a verified, production-tested software asset repository containing:

- An optimized database access and connection-pooling subsystem (DB_Pool_v3).
- A certified cryptographic security and token-hashing library (CryptoLib_AES).
- An algorithmic tax and multi-currency exchange calculation engine (TaxEngine_Global).

Task Requirements:

1. Evaluate whether the engineering team should adopt a **Top-Down Design**, **Bottom-Up Design**, or **Hybrid Design** strategy for this enterprise system.
2. Formulate a detailed engineering rationale for the chosen strategy, explicitly addressing requirements alignment, time-to-market, and risk management.
3. Outline the practical phase-by-phase architectural execution plan.

Model Engineering Response:

1. Recommended Strategy:

A Hybrid Design Strategy is a reasonable recommendation **for this scenario.**

2. Technical & Economic Rationale:

- **Inadequacy of Pure Top-Down Design:** A strict top-down strategy begins from the root system coordinator and decomposes purely according to high-level functional decomposition, ignoring pre-existing code until leaf modules are reached. In this scenario, pure top-down design could design interfaces before accounting for the signatures of existing production libraries (DB_Pool_v3, CryptoLib_AES, TaxEngine_Global), which may require adapters or redesign if the interfaces do not align.
- **Inadequacy of Pure Bottom-Up Design:** A bottom-up start can make it harder to keep system-wide requirements visible unless the architecture is checked against those requirements. In this scenario, logistics workflows and regulatory rules need to guide the organization of the reused components.
- **The Hybrid Architectural Advantage:** Hybrid design simultaneously executes top-down decomposition of business requirements into major functional subsystems while bottom-up integrating and adapting proven, pre-tested software assets. This strategy can relate system-level requirements to the available components; the actual benefits depend on component fit, interfaces, and verification.

3. Phase-by-Phase Execution Plan:

1. **Phase 1 (Top-Down Requirements Partitioning):**
2. Decompose the logistics workflow into four primary subsystems: *Order Ingestion*, *Inventory Allocation*, *Billing & Tax*, and *Audit Dispatch*.
3. **Phase 2 (Bottom-Up Asset Adaptation & Wrapping):**
4. Create clean facade or adapter wrappers around DB_Pool_v3, CryptoLib_AES, and TaxEngine_Global to ensure their interfaces conform to the new architectural standard.
5. **Phase 3 (Mid-Level Integration & Verification):**
6. Connect top-level subsystem coordinators to the adapted bottom-level assets via well-defined interface contracts, executing early integration testing using lightweight stubs for missing components.

> *Author-created scenario informed by K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–215; Companion Slides 32–35. It is not a textbook exercise.*

> **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - في الأنظمة المؤسسية الكبيرة المعقدة التي تجمع بين متطلبات عمل جديدة ومكتبات برمجية سابقة (Reused Libraries)، فإن الخيار الهندسي العملي هو التصميم الهجين (Hybrid Design). التفكيك التنازلي يحافظ على متطلبات العمل العليا، بينما التجميع التصاعدي يستثمر المكونات المختبرة ويوفر شهوراً من العمل.

Q5.S03. Evaluating Enterprise Banking Architecture Against Bertrand Meyer's Five Modularity Criteria

Engineering Scenario:

A core banking payment processing gateway handles cross-border transactions, currency conversion, fraud detection, and regulatory compliance logging.

Task Requirements:

Evaluate this banking architecture against all five of **Bertrand Meyer's Modularity Criteria** (Decomposability, Composability, Understandability, Continuity, and Protection). For each criterion, specify a concrete architectural mechanism that ensures compliance.

Model Engineering Response:

1. **Modular Decomposability:**
2. *Evaluation Criterion:* Can the complex payment gateway be systematically partitioned into smaller, independently solvable sub-problems?
3. *Architectural Mechanism:* Decompose the gateway into distinct autonomous services: PaymentAuthorizationService, CurrencyExchangeService, FraudDetectionService, and RegulatoryAuditService. Each service can be assigned to a separate development squad, implemented in parallel, and verified independently.
4. **Modular Composability:**
5. *Evaluation Criterion:* Can modules engineered for this gateway be freely reused to construct other banking systems?
6. *Architectural Mechanism:* Design CurrencyExchangeService and FraudDetectionService as self-contained components with clean REST/gRPC interfaces and zero coupling to internal database tables. The currency exchange module can subsequently be composed into the bank's retail mobile app or wealth management portal without modification.
7. **Modular Understandability:**
8. *Evaluation Criterion:* Can an engineer inspect and comprehend an individual module without studying the internal implementations of companion modules?

9. *Architectural Mechanism:* Strict interface contracts and encapsulation. An engineer maintaining PaymentAuthorizationService only needs to read its local logic and the published method signatures of the fraud service (e.g., `is_fraudulent(transaction) -> bool`); they never need to understand the machine learning neural network weights inside the fraud detection engine.
10. **Modular Continuity:**
11. *Evaluation Criterion:* Does a minor modification to business requirements trigger changes in only one or a minimal cluster of modules?
12. *Architectural Mechanism:* If the international banking regulator introduces a new value-added tax (VAT) rule for European transactions, the change is entirely confined to TaxEngine_Global. The core authorization engine, fraud detector, and messaging queues may remain unchanged if the architecture isolates the tax rule and its interfaces.
13. **Modular Protection:**
14. *Evaluation Criterion:* If a runtime fault or unhandled exception occurs inside a module, is it strictly contained without causing catastrophic system failure?
15. *Architectural Mechanism:* Implementation of the *Circuit Breaker* and *Defensive Boundary* patterns. If an external currency exchange provider experiences a network timeout or crash, CurrencyExchangeService catches the exception locally, emits a fallback cached exchange rate or controlled error code, and prevents the crash from taking down the core banking database.

Source: Pressman (5th ed.), §13.4.3, printed p. 345 / PDF p. 373; K.K. Aggarwal & Yogesh Singh pp. 206–207. Exercise 5.4

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- معايير ماير الخمسة في الأنظمة الحقيقية:
- Decomposability ← تقسيم النظام إلى خدمات مستقلة.
- Composability ← تصميم وحدات قابلة للتركيب في أنظمة أخرى دون تعديل.
- Understandability ← فهم الوحدة من واجهتها دون قراءة كود الوحدات الأخرى.
- Continuity ← حصر أثر تعديل القوانين المالية داخل وحدة الضرائب وحدها.
- Protection ← عزل الاستثناءات عبر قواطع الدائرة (Circuit Breakers) لمنع انهيار النظام.

Section 2: Coupling & Cohesion Diagnostics and Refactoring

Q5.S04. Architectural Coupling & Cohesion Diagnostic and Refactoring Case

Engineering Scenario:

You are conducting an architectural audit on four existing code modules in an enterprise software system:

- **Module A (AuditLogManager & AuthModule):** The AuditLogManager accesses the internal memory pointer of AuthModule, modifying its private boolean variable is_authenticated directly via memory offset to bypass expired session tokens during batch testing.
- **Module B (InventoryController & OrderProcessor):** Both modules communicate by reading and writing to an unencapsulated, public global struct named g_shared_order_buffer defined in a shared C header file.
- **Module C (StatisticalCalculator):** Contains a single procedure calculate_variance(float* samples, int count) that performs exclusively the mathematical calculation of sample variance and returns a float.
- **Module D (UtilityDispatcher):** Contains a function execute_command(int op_code, void* data) that executes a print routine if op_code=1, initializes hardware ports if op_code=2, and flushes transaction logs if op_code=3.

Task Requirements:

1. Diagnose and state the exact Coupling Level and Cohesion Level for each module.
2. Identify the specific architectural hazards and failure modes of Modules A, B, and D.
3. Provide concrete refactoring recommendations to restore optimal software engineering quality.

Model Engineering Response:

1. Diagnostic Evaluation:

- **Module A:**
- *Coupling:* **Content Coupling (Worst / Pathological).**
- *Cohesion:* Procedural / Coincidental.
- **Module B:**
- *Coupling:* **Common Coupling.**
- *Cohesion:* Communicational.
- **Module C:**
- *Coupling:* **Data Coupling (Loosest / Most Desirable).**
- *Cohesion:* **Functional Cohesion (Strongest / Most Desirable).**
- **Module D:**
- *Coupling:* **Control Coupling.**
- *Cohesion:* **Logical Cohesion.**

2. Architectural Hazards & Failure Modes:

- **Module A (Content Coupling):** Utter violation of encapsulation. Any compiler optimization, variable reordering, or refactoring inside AuthModule will corrupt memory in AuditLogManager, leading to segmentation faults or catastrophic security vulnerabilities.
- **Module B (Common Coupling):** Shared global state makes tracking mutations impossible. If an erroneous null pointer is written to g_shared_order_buffer, debugging cannot determine which of the two modules caused the corruption. Concurrent access may also require synchronization; a race condition is possible if shared state is accessed without suitable coordination.

- **Module D (Logical Cohesion / Control Coupling):** Callers must know internal op_code mappings. Modifying hardware initialization risks breaking the printing logic because both share local variables inside the same nested switch block.

3. Architectural Refactoring Recommendations:

1. **Refactoring Module A:** Eliminate memory manipulation completely. Provide a secure, explicit public API method in AuthModule (e.g., bool revalidate_session(SessionToken token)), passing atomic parameters to achieve **Data Coupling**.
2. **Refactoring Module B:** Encapsulate the shared data inside an autonomous OrderRepository class. Hide state behind synchronized getter/setter methods (add_order(), get_pending_orders()), transforming common coupling into clean **Data/Stamp Coupling**.
3. **Refactoring Module D:** Decompose UtilityDispatcher into three separate, functionally cohesive procedures: print_report(), initialize_hardware(), and flush_transaction_logs(). This removes the selector flag from the proposed design.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 207–213; Companion Slides 15–30. Exercises 5.5, 5.6, 5.8, 5.9, 5.10

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- في أسئلة تشخيص الاقتران والتماسك:
- التعديل المباشر على الذاكرة والمتغيرات الخاصة = Content Coupling (كارثي).
- القراءة والكتابة في متغيرات عامة مشتركة = Common Coupling (خطير).
- استخدام علامات ومفاتيح switch لاختيار إحدى الوظائف = Logical Cohesion & Control Coupling.
- إجراء عملية رياضية واحدة بمتغيرات ذرية = Functional Cohesion & Data Coupling (النموذج المثالي).

Q5.S05. Object-Oriented Refactoring: Eliminating a "God Class" Anti-Pattern

Engineering Scenario:

An architectural review of a legacy web application discovers a bloated monolithic class named UserManager containing 42 methods and 1,800 lines of code. An inspection of its responsibilities reveals that it handles:

1. User authentication and cryptographic password hashing.
2. SQL queries for user database CRUD operations.
3. Composing and dispatching marketing and password-reset emails.
4. Generating and formatting PDF user activity audit reports.
5. Managing active user session tokens in Redis.

Task Requirements:

1. Identify the fundamental Object-Oriented Design principles violated by UserManager.
2. Formulate a refactored object-oriented architecture that redistributes these responsibilities into cohesive, loosely coupled classes.
3. Provide the UML class structural inventory and interaction contracts for the refactored design.

Model Engineering Response:

1. Architectural Violations:

- **Violation of the Single Responsibility Principle (SRP):** UserManager has at least five distinct reasons to change (changes to database schemas, password hashing algorithms, email SMTP protocols, PDF formatting styles, or session storage engines).
- **Severe Low Class Cohesion:** The internal methods do not operate on a common state; PDF formatting methods have zero overlap with password hashing logic.
- **Extreme Fragility & Coupling:** Any modification to email templates risks destabilizing the database persistence logic.

2. Refactored Object-Oriented Architecture:

Decompose the monolithic "God Class" into five cohesive, cooperating classes, coordinated by a clean facade:

1. **User (Domain Entity):**
2. *Responsibility:* Pure domain state (ID, name, email, hashed credentials, account status).
3. *Cohesion:* High functional domain cohesion.
4. **UserRepository (Persistence Layer):**
5. *Responsibility:* SQL CRUD operations (save(User), find_by_id(UUID), update(User)). Encapsulates database queries.
6. **AuthenticationService (Domain Logic):**
7. *Responsibility:* Validating credentials, coordinating password hashing via a separate PasswordEncoder component, and generating JWT tokens.
8. **NotificationService (Infrastructure Service):**
9. *Responsibility:* Assembling email templates and dispatching messages via SMTP/API (send_welcome_email(User), send_reset_token(User， Token)).
10. **UserReportGenerator (Reporting Service):**
11. *Responsibility:* Extracting audit logs and rendering formatted PDF reports (generate_audit_pdf(User) -> byte[]).

3. Engineering Benefits:

- Each class now possesses a single, well-defined responsibility.
- Database technologies can be refactored inside UserRepository without touching AuthenticationService or NotificationService.
- Unit testing is simplified: mock repositories can be injected into AuthenticationService to test login flows in milliseconds without a live SQL database.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 228–230; Companion Slides 69–71. Exercise 5.20

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - صنف الإله (God Class) هو فخ برمجي شهير يجمع مسؤوليات متباينة (قواعد بيانات، إيميل، تشفير، تقارير PDF). الحل المعماري هو تفكيكه فوراً وفق مبدأ المسؤولية الواحدة (SRP) إلى أصناف مستقلة: كيان النطاق (User)، مستودع البيانات (Repository)، خدمة التوثيق (AuthService)، وخدمة الإشعارات (EmailService).

Section 3: Structural Metrics & Formal Review Protocols

Q5.S06. Structural Hierarchy Refactoring Using Fan-In and Fan-Out Metrics

Engineering Scenario:

An architectural metric analysis of a complex billing structure chart yields the following anomalies:

- **Anomaly 1:** The top coordinator module BillingExecutive directly invokes 14 different subordinate modules representing low-level file formatters, tax lookups, socket senders, and printer drivers ($\text{Fan-Out} = 14$).
- **Anomaly 2:** An intermediate business logic module ComputeRegionalSurcharge is directly invoked by 9 different high-level coordinators spanning three separate subsystems ($\text{Fan-In} = 9$ at middle level).
- **Anomaly 3:** A low-level string-sanitization routine clean_alphanumeric_str is invoked by only 1 module ($\text{Fan-In} = 1$ at leaf level), while an identical helper function exists under another name in four other modules.

Task Requirements:

1. Analyze the structural defects indicated by these fan-in and fan-out metrics.
2. Apply architectural restructuring heuristics to normalize the call hierarchy.

Model Engineering Response:

1. Structural Defect Analysis:

- **Excessive Top-Level Fan-Out ($\text{Fan-Out} = 14$):**
- Violates the standard cognitive span-of-control heuristic ($3 \leq \text{Fan-Out} \leq 7$).
- Indicates that BillingExecutive is micromanaging low-level operational details rather than delegating to intermediate abstraction layers. The module suffers from excessive control coupling and high mental complexity.
- **Inverted Middle-Level Fan-In ($\text{Fan-In} = 9$ at Middle Layer):**
- High fan-in is desirable at leaf utility levels, but problematic at the middle layer. It indicates that multiple disparate coordinators across different functional domains are tightly coupled to a specific business logic routine, creating cross-subsystem tangling and making future modifications to surcharge rules hazardous.

- **Fragmented Leaf Fan-In ($\text{Fan-In} = 1$ with Duplication):**
- Indicates missed reuse opportunities and code duplication across the system.

2. Architectural Restructuring Heuristics:

1. **Factoring BillingExecutive (Normalizing Fan-Out):**
2. Introduce intermediate coordinator modules to manage specific functional domains:
    - InvoicePreparationManager (governs data gathering and formatting).
    - TaxAndSurchargeManager (governs tax calculations).
    - DispatchManager (governs printing and network transmission).
3. This reduces the fan-out of BillingExecutive from 14 down to 3, restoring a balanced, manageable control hierarchy.
4. **Encapsulating ComputeRegionalSurcharge:**
5. Relocate regional surcharge calculations under a unified TaxCalculationSubsystem. External coordinators should interact with the subsystem facade rather than invoking internal procedural rules directly, isolating business rule variations.
6. **Consolidating Leaf Utilities (Maximizing Leaf Fan-In):**
7. Eliminate duplicate helper routines and establish a single, shared StringUtility module containing clean_alphanumeric_str().
8. Promote this routine to a high fan-in ($Fan\text{-}In \geq 5$) leaf service, maximizing code reuse and ensuring centralized maintenance.

Source: Pressman (5th ed.), §13.4.5, printed pp. 347–348 / PDF p. 375; Rajib Mall (4th ed.), §5.4, printed pp. 300–301.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- قواعد تحسين مروحة الدخول ومروحة الخروج (Fan-In / Fan-Out):
- إذا تجاوزت مروحة الخروج للوحدة العليا 7 مستدعين (Fan-Out > 7)، فهذا يعني فشلاً في التجريد ويجب إدخال طبقة وسطى من المديرين الفرعيين.
- ارتفاع مروحة الدخول (High Fan-In) ممتاز ومطلوب في طبقة الخدمات الدنيا (Leaf Utilities) لأنه يدل على إعادة الاستخدام، ولكنه خطر في الطبقات الوسطى لأنه يدل على تشابك غير مبرر في منطق الأعمال.

Q5.S07. Formal Software Design Review Simulation: IEEE Std 1016 SDD Inspection

Engineering Scenario:

A software engineering team has completed the draft Software Design Description (SDD) conforming to IEEE Std 1016-1998 for an Automated Patient Medication Dispenser. Before implementation begins, a Formal Technical Review (Architecture Inspection) is convened.

Task Requirements:

1. Detail the formal roles of participants in the Design Review meeting.
2. Provide the essential 5-point Design Inspection Checklist used by reviewers.
3. Construct a simulated Design Review Issue Log capturing defects identified during the review.

Model Engineering Response:

1. Formal Review Participant Roles:

- **Moderator (Review Leader):** Chairs the meeting, ensures rigorous adherence to review procedures, prevents personal criticism, and manages time.
- **Author (Lead Architect):** Presents the SDD structure, provides rationale for design choices, and answers technical inquiries.
- **Inspectors (Peer Reviewers):** Senior software engineers and domain specialists who independently scrutinized the SDD prior to the meeting.
- **Requirements Analyst:** Verifies complete forward and backward traceability against the approved SRS.
- **Scribe (Recorder):** Objectively records every detected defect, discrepancy, and action item on the official Review Issue Log.

2. The 5-Point Design Inspection Checklist:

1. **Traceability & Completeness:** Does every design entity map to at least one requirement in the SRS, and are all SRS functional/performance requirements accounted for?
2. **Modularity & Independence:** Do all modules exhibit high functional cohesion? Are harmful coupling levels (content, common, control) strictly avoided?
3. **Interface Specification:** Are all ten IEEE Std 1016 entity attributes defined? Are parameter data types, directionality, ranges, and error return codes fully specified?
4. **Defensive Error & Fault Handling:** Does the design specify explicit responses for edge cases, invalid sensor inputs, power interruptions, and hardware failures?
5. **Resource & Algorithmic Feasibility:** Are memory footprints, execution times, and communication bandwidth within projected hardware constraints?

3. Simulated Formal Design Review Issue Log:

Issue ID	Design Entity / Section	Defect Description	Severity	Assigned Actionee
REV-01	DispenserController (SDD §4.2)	**Common Coupling:** Directly accesses a shared global memory struct g_dosage_state instead of using validated interface accessors.	**Critical**	Lead Architect (Redesign interface)
REV-02	SensorInterface (SDD §4.5)	**Omission:** Lacks explicit timeout specifications and exception handling for unresponsive weight sensors.	**Major**	Embedded Engineer (Add timeout handler)
REV-03	PatientRecordView (SDD §3.1)	**Traceability Gap:** Includes an unauthorized biometric facial recognition module not requested in the SRS ("Gold Plating").	**Moderate**	Requirements Analyst (Remove module)

Issue ID	Design Entity / Section	Defect Description	Severity	Assigned Actionee
REV-04	DosageCalculator (SDD §4.8)	**Interface Ambiguity:** Parameter patient_weight does not specify measurement units (kilograms vs. pounds).	**Critical**	Algorithm Designer (Enforce SI units)

Source: Rajib Mall (4th ed.), §6.7, printed pp. 350–351; IEEE Std 1016-1998; Pressman (5th ed.), §13.2.

📝 ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مراجعة التصميم الرسمية (Formal Design Review) تركز على كشف العيوب البنيوية قبل كتابة الكود:
- التأكد من مطابقة الـ SDD لوثيقة المتطلبات (SRS).
- حظر البرمجة الزائدة غير المطلوبة (Gold Plating).
- فحص استقلالية الوحدات وخلوها من Common Coupling.
- تسجيل كل الملاحظات رسمياً في سجل العيوب (Review Issue Log) وتحديد المسؤولية وتاريخ الإغلاق.

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Concepts Covered
1. Direct Exercise 5.12: Reuse Strategy	Q5.S01	K.K. Aggarwal pp. 213–214; Slide 33	Bottom-Up Design defined by starting from existing lower-level library modules.
2. Complex Enterprise Strategy	Q5.S02	K.K. Aggarwal pp. 213–215; Slides 32–35	Top-Down vs Bottom-Up vs Hybrid selection with reusable assets in enterprise supply chains.
3. Meyer's Modularity Assessment	Q5.S03	Pressman §13.4.3; K.K. Aggarwal pp. 206–207	Decomposability, composability, understandability, continuity, protection applied to banking systems.
4. Coupling/Cohesion Diagnostics	Q5.S04	K.K. Aggarwal pp. 207–213; Slides 15–30	Diagnostic evaluation of Content, Common, Control, and Data coupling; refactoring strategies.
5. Object-Oriented Refactoring	Q5.S05	K.K. Aggarwal pp. 228–230; Slides 69–71	"God Class" decomposition, Single Responsibility Principle, decoupling persistence from domain logic.
6. Hierarchy Metrics Optimization	Q5.S06	Pressman §13.4.5; Mall §5.4	Fan-out normalization ($3 \leq \text{Fan-Out} \leq 7$), intermediate coordinator factoring, maximizing leaf fan-in.
7. Formal Architecture Inspection	Q5.S07	Mall §6.7; IEEE Std 1016-1998; Pressman §13.2	Review roles, 5-point inspection checklist, formal Design Review Issue Log construction.
